

LAD-GENERIC: PARAMETER-EFFICIENT CONVERSION OF INSTRUCTION-TUNED LANGUAGE MODELS TO DIFFUSION

Anonymous authors

Paper under double-blind review

ABSTRACT

Masked diffusion language models permit parallel token updates and flexible generation orders, but leading systems rely on costly pretraining or continued full-model training. We ask whether diffusion generation can instead be added to existing instruction-tuned autoregressive models using a single parameter-efficient conversion recipe. Build Your Own DLM (BYO-DLM) replaces causal attention with bidirectional attention and trains only query/value LoRA adapters under a masked denoising objective, leaving 94.2–95.9

1 INTRODUCTION

Autoregressive (AR) language models generate a sequence from left to right. This method has enabled effective scaling, but it fixes the generation order, limiting the model to unidirectional attention. Masked diffusion language models instead begin with a corrupted sequence and iteratively reconstruct multiple positions using bidirectional context (Austin et al., 2021a; Sahoo et al., 2024; Shi et al., 2024). Their number of network evaluations can be chosen independently of output length, and early erroneous predictions can be revised. However, training or adapting models for diffusive generation has thus far been a computationally intensive and expensive procedure.

The authors of LLaDA were the first to show that masked diffusion can scale to an 8B-parameter general-purpose language model (Nie et al., 2025). While they based the architecture on the Llama 3.1 8B model, they pre-trained it from scratch using 2.3T tokens and 130,000 GPU-hours. DiffuLLaMA demonstrated a more efficient route, converting a similarly sized pre-trained AR model through continued diffusion training (Gong et al., 2025). Since then, various methods have been proposed to improve efficiency and task retention. Dream explicitly builds on DiffuLLaMA’s AR initialization and adds context-adaptive token-level noise rescheduling (Ye et al., 2025); Efficient-DLM combines blockwise attention with position-dependent masking (Fu et al., 2025); and iLLaDA returns to from-scratch diffusion pretraining at substantially larger scale (Nie et al., 2026). LAD was the first to show that low-rank adaptation (LoRA) can induce diffusion generation with substantially fewer trainable parameters (Kuiper et al., 2025; Hu et al., 2022). It remains unclear how reliably this minimal recipe transfers across architecture families and how much capability is retained relative to each model’s own AR parent.

Our aim is not merely to avoid updating every parameter, but to provide a practical alternative to training a separate diffusion foundation model for every model family. Reusing existing AR knowledge reduces the incremental data, accelerator count, and financial barrier to experimentation. This emphasis follows calls to treat computational efficiency and access as research outcomes alongside accuracy (Strubell et al., 2019). It also motivates a sustainability benefit, although compute volume alone is not a carbon measurement: emissions depend strongly on hardware, datacenter efficiency, location, and energy supply (Patterson et al., 2021). We therefore report bounded incremental training scale and hardware while reserving quantitative environmental claims for future energy measurements.

We study this question with LAD-Generic, a generalizable conversion and evaluation pipeline for four instruction-tuned decoder families: Gemma 2 9B, Llama 3.1 8B, Qwen2.5 7B, and Ministral 8B. At the time the experimental design was frozen, we selected the latest openly available text-only

instruction checkpoint from each chosen family at a comparable nominal scale that fit the same single-GPU pipeline. Excluding multimodal variants avoids vision components and modality-specific training as additional confounders. Every backbone receives the same bidirectional attention intervention, LoRA target modules, and masked-diffusion objective. This paired design separates conversion quality from the strength of the starting checkpoint. We directly evaluate LLaDA 8B Instruct because its standard masked-diffusion objective, instruction-tuning setup, and overlapping benchmark suite make it the cleanest external control. We additionally report published DiffuLLaMA, Dream-Instruct, iLLaDA-Instruct, and Efficient-DLM values, but keep them protocol-separated: their additional attention, masking, scheduling, data, and inference techniques would obscure rather than strengthen a controlled comparison.

Our contributions are:

- a single, architecture-agnostic recipe that converts four instruction-tuned AR model families to masked diffusion using only Q/V LoRA parameters on one Google Colab GPU;
- a paired evaluation of each diffusion model and its frozen AR parent across general knowledge, reasoning, mathematics, and code benchmarks;
- a controlled analysis of base-model choice, 25k versus 50k updates, denoising-step count, validation loss, and externally scored generation perplexity; and
- a transparent comparison between results produced by our common harness and the published values reported with the LLaDA evaluation protocol.

2 RELATED WORK

Discrete diffusion models corrupt and reconstruct categorical sequences rather than adding Gaussian noise (Austin et al., 2021a). Recent masked formulations simplify this process to absorbing-state masking and yield tractable likelihood-based objectives (Sahoo et al., 2024; Shi et al., 2024). LLaDA scaled this formulation to language-model pretraining and instruction following (Nie et al., 2025); iLLaDA retains full bidirectionality but scales pretraining to 12T tokens and uses a 25B-token instruction corpus for 12 epochs (Nie et al., 2026).

AR-to-diffusion conversion amortizes the knowledge already acquired by an AR checkpoint. DiffuLLaMA established this route through full-model continued pretraining, using 65B tokens at 7B scale (Gong et al., 2025). Dream does not independently introduce AR initialization: it explicitly adopts DiffuLLaMA’s shifted-prediction initialization, continues Qwen2.5 for 580B tokens, adds context-adaptive token-level noise rescheduling, and then tunes on 1.8M instruction pairs for three epochs (Ye et al., 2025). Efficient-DLM preserves causality across blocks, exposes clean preceding blocks, and biases masking toward later positions; its 8B model receives 500B tokens of continued training (Fu et al., 2025). These methods target stronger task retention and/or faster decoding, whereas LAD-Generic deliberately tests a smaller intervention: fully bidirectional attention and Q/V-only LoRA on instruction data.

Hybrid and blockwise models offer a different compromise between bidirectionality and deployability. Block Diffusion is autoregressive across blocks but diffusive within each block, enabling variable-length generation and inter-block KV caching (Arriola et al., 2025). Fast-dLLM v2 converts Qwen2.5 models with approximately 1B fine-tuning tokens and combines blockwise complementary masking with hierarchical caching (Wu et al., 2025). Sequential Diffusion Language Models (SDLM) instead learn next-sequence prediction and dynamically finalize variable-length subsequences, reporting conversion with 2.3 billion tokens (Liu et al., 2025). These systems retain more left-to-right structure than our fully bidirectional answer block, making them especially relevant to speed, caching, and arbitrary-length generation but not suited as direct controls for our intervention.

Several recent studies have used LoRA for adaptation of diffusion models for related but fundamentally different goals. Uno freezes an AR model and learns lightweight LoRA diffusion-draft weights by consistency distillation, then uses the unchanged AR model as a verifier; its Ψ -Spec sampler therefore provides lossless acceleration of the AR distribution rather than replacing it with a standalone diffusion distribution (Sahoo et al., 2026). The dLLM framework standardizes training, inference, and evaluation across LLaDA- and Dream-style models. Importantly, its rank-128 LoRA experiment is reasoning SFT of these already-diffusive checkpoints, not LoRA-based AR-to-diffusion conversion.

Table 1: Reported training scale for relevant large diffusion systems. “Full” means the main diffusion model weights are updated; LoRA denotes parameter-efficient diffusion weights. Hardware generations and sequence lengths differ, so GPU-hours are not interchangeable. NR: not reported.

System	Starting point	Diffusion-stage data	Updated weights	Hardware	Reported time
LAD-Generic 25k (ours)	instruction AR	$\leq 102.4\text{M}$ tokens	Q/V LoRA	1 $\times$ RTX Pro 6000 (Colab)	3.2 GPU-h
LAD-Generic 50k (ours)	instruction AR	$\leq 204.8\text{M}$ tokens	Q/V LoRA	1 $\times$ RTX Pro 6000 (Colab)	6.4 GPU-h
DiffuLLaMA 7B	LLaMA2 7B	65B tokens	Full	64 $\times$ GH200	NR
LLaDA 8B	random init.	2.3T tokens + 4.5M SFT pairs	Full	H800	130,000 GPU-h (pretrain)
Dream 7B	Qwen2.5 7B	580B tokens + 1.8M pairs $\times 3$	Full	NR	NR
iLLaDA 8B	random init.	12T tokens + 25B-token SFT $\times 12$	Full	NR	NR
Efficient-DLM 8B	Qwen3 8B	500B tokens	Full	128 $\times$ H100	NR
Uno 8B	frozen AR	7B distillation tokens	LoRA diffusion weights	64 $\times$ H200	3,840 GPU-h
dLLM reasoning SFT	LLaDA/Dream (DLM)	s1K $\times 20$ epochs	rank-128 LoRA	8 $\times$ A100	NR
dLLM Tiny-A2D	Qwen3 0.6B AR	instruction mixture $\times 10$ epochs	NR (not LoRA-reported)	64 $\times$ A100	NR
Fast-dLLM v2 7B	Qwen2.5 7B AR	$\sim 1\text{B}$ tokens	Full	64 $\times$ A100	768 GPU-h
SDLM	instruction AR	2.3B tokens	NR	NR	NR

Its separate Tiny-A2D experiment converts Qwen3-0.6B with instruction SFT on 64 A100 GPUs and does not report LoRA as the conversion mechanism (Zhou et al., 2026). NaRA likewise uses noise-conditioned LoRA to adapt existing diffusion LLMs rather than to induce diffusion behavior in an AR checkpoint (Wang et al., 2026). These distinctions isolate our central question: can low-rank updates themselves perform the large-model paradigm conversion?

Table 1 places the training budgets of related studies side by side. For conversion methods, these budgets exclude the substantial earlier cost of training the AR parent; for from-scratch methods they include diffusion pretraining. Consequently, the table compares the additional cost required to obtain diffusion behavior, not total lifecycle compute.

Our study has explicitly chosen to hold the adaptation mechanism as constant as the architectures permit, repeat it across four families, and benchmark each converted model against the exact AR checkpoint from which it originated. This paired comparison is essential because it allows us to compare our results directly with the prior capability of the parent model.

3 METHOD

3.1 CONDITIONAL MASKED DENOISING

Let p be an observed instruction prompt and $x_0 = (x_0^1, \dots, x_0^L)$ its clean response region. For each example, we sample $t \sim \mathcal{U}[10^{-3}, 1]$ and independently replace each eligible response token with a single tokenizer-native mask representation with probability t . Denote the resulting sequence by x_t and the sampled mask positions by M_t . The model is trained with same-position cross-entropy,

$$\mathcal{L}(\theta) = -\mathbb{E}_{t, x_0, x_t} \left[\frac{1}{t|E|} \sum_{i \in M_t} \log p_{\phi, \theta}(x_0^i | p, x_t) \right], \quad (1)$$

where E is the eligible answer-and-padding region, ϕ denotes the frozen parent parameters, and θ the LoRA parameters. Prompt and chat-template tokens are always visible and never enter the loss. Training corruption is resampled online; validation corruption is deterministic from the example index and seed.

The clean terminating EOS is included as a genuine answer token. To teach the fixed-width generator to emit termination beyond the answer, dynamic batch padding is represented by repeated EOS tokens; these positions are also eligible for masking and loss. Thus padding is part of the learned denoising target rather than conventional ignored padding. This design choice should be interpreted separately from the semantic answer-token objective and is ablated in **XXX: appendix/table after final ablation selection**.

3.2 BIDIRECTIONAL CONVERSION AND TRAINABLE PARAMETERS

We replace the causal mask with an all-zero four-dimensional additive attention mask, so every prompt, answer, and EOS-padding position can attend to every position. The architecture and output vocabulary remain unchanged. We attach LoRA adapters to every self-attention query and value projection, with rank $r = 1024$, scaling $\alpha = 1024$, and dropout zero. Backbone weights, token

Table 2: Converted parent checkpoints and audited trainable parameter counts. All trainable parameters are Q/V LoRA weights.

Parent checkpoint	Total parameters	Trainable	Trainable (%)
Gemma 2 9B IT	10.732B	572.5M	5.33
Llama 3.1 8B Instruct	8.466B	436.2M	5.15
Qwen2.5 7B Instruct	7.939B	323.0M	4.07
Ministral 8B Instruct 2410	8.511B	490.7M	5.77

embeddings, the language-model head, biases, and normalization layers are frozen. Each model reuses an existing single-token string from its tokenizer (MASK for Gemma, Llama, and Qwen; <?> for Ministral), avoiding vocabulary expansion. At experiment-design time, these were the latest openly available text-only instruction checkpoints from the four selected families at a comparable nominal scale; using text-only variants keeps the conversion interface uniform and avoids multimodal components as a confounder.

Table 2 reports the parameter audits saved by the training runs. The trainable fraction varies because the four attention implementations have different projection shapes and layer counts.

After training, we fold each LoRA branch into its parent projection, $W_{\text{merged}} = W + (\alpha/r)BA$, and remove the adapter modules. The resulting standalone checkpoint retains bidirectional inference but has the base architecture’s parameter count: Llama decreases from 8.466B adapter-loaded parameters to 8.030B merged parameters (16.06 GB in BF16). Thus the model-size increase is temporary, although optimizer states and activations still add training-time memory.

3.3 TRAINING DATA

All main models use Ruurd/LAD-training-1m-256. The training pipeline shuffles the source with seed 42 and consumes 400,000 usable examples per 25,000-update stage. Examples are retokenized with each parent model’s native chat template and tokenizer. Sequences are dynamically padded to a multiple of 16 and limited to 256 tokens; only the current answer region is corrupted. The dataset builder groups sources into general instruction following, multiple-choice reasoning, mathematics, and code, deduplicates normalized prompts, and blocks exact normalized matches with represented benchmark validation/test prompts. **XXX: Archive the exact Hub revision and manifest used by these original runs, then insert the final category/source counts and long-answer filtering policy.**

The four 25k models see the same seed-selected 400k-example prefix. For the 50k Llama condition, the 25k final adapter initializes a second run over the next 400,000 examples from the same stable dataset shuffle. The second stage resamples corruption masks and restarts AdamW and its learning-rate schedule; it is therefore continued adapter training, not a restoration of optimizer state.

3.4 OPTIMIZATION

Every run uses batch size 16 with no gradient accumulation, AdamW, learning rate 10^{-5} , weight decay 0.01, gradient clipping at 0.5, 100 warmup updates, and cosine decay. The maximum is 25,000 optimizer updates per stage. Training uses BF16 master weights and Transformer Engine HYBRID FP8 execution; evaluation runs in BF16. All reported conversions were executed in a single-GPU Google Colab runtime using one NVIDIA RTX PRO 6000 Blackwell Server Edition, with no multi-GPU or multi-node parallelism. The code, memory strategy, sequence length, and adapter scope were explicitly designed around this constraint. We save model checkpoints every 1,000 updates (5,000 in the continued stage), evaluate denoising loss every 250 updates on 200 deterministic validation examples, and use seed 42. **XXX: Insert wall-clock duration, GPU-hours, peak memory, Colab tier, and software versions.**

3.5 DIFFUSION GENERATION

Generation begins from a fully masked answer block. At each reverse step the model predicts every masked position, after which the least-confident positions are retained for reconsideration according

Table 3: Current task budgets for the shared LLaDA-style diffusion evaluator. These must be verified against the final saved benchmark manifests.

Task	MMLU	MMLU-P	HSwag	ARC-C	GSM8K	MATH	GPQA	HEval	MBPP
Tokens/steps	64	128	64	64	256	256	128	256	128

to the LLaDA fixed-transfer schedule; newly accepted positions become permanent context. For the primary benchmark evaluation we use greedy token selection, low-confidence remasking, and task-specific output/step budgets (Table 3). Open-ended evaluation generates 128 tokens with 64 denoising steps, temperature 0.7, and delayed retention of EOS/EOT predictions. **XXX: Confirm these settings against the immutable benchmark run manifests associated with every workbook column.**

4 EVALUATION

4.1 PAIRED MODELS AND BASELINES

For Gemma, Llama, Qwen, and Ministral we compare the diffusion adapter with greedy generation from its unchanged AR parent. LLaDA 8B Instruct is evaluated as an external diffusion baseline. We report two explicitly separated LLaDA comparisons: (i) LLaDA run through our prompts, sampling code, subsets, and scorers; and (ii) values transcribed from the LLaDA paper/evaluation suite. The latter are contextual references rather than directly matched measurements.

4.2 BENCHMARKS AND SCORING

Our validation suite contains MMLU, MMLU-Pro, HellaSwag, ARC-Challenge, GSM8K, MATH-500, GPQA-Main, HumanEval, and sanitized MBPP (Hendrycks et al., 2021a; Wang et al., 2024; Zellers et al., 2019; Clark et al., 2018; Cobbe et al., 2021; Hendrycks et al., 2021b; Rein et al., 2023; Chen et al., 2021; Austin et al., 2021b). HellaSwag uses its labelled validation split, ARC-Challenge and MATH-500 use test, and GPQA-Main uses the release’s train-named evaluation split. Multiple-choice answers are extracted by label; GSM8K uses the final numeric answer; MATH uses normalized boxed answers; and HumanEval/MBPP execute tests in isolated local subprocesses. The current workbook records 50 examples per task unless marked †, where 250 were evaluated. This is a preliminary subset evaluation; the final paper will report **XXX: full-set results or a single predeclared sampling design, with uncertainty intervals.**

For 30 fixed open-ended prompts, Microsoft Phi-4 scores each generated answer autoregressively. Our primary aggregate is token-weighted perplexity, $\exp(\sum_j \text{NLL}_j / \sum_j N_j)$, so each scored token contributes equally; per-answer means and medians are retained as supplementary diagnostics. These are external-reference fluency measures, not likelihoods of the diffusion model itself. This distinction is important: generative perplexity inherits the reference model’s preferences and does not penalize loss of sample diversity (Turok et al., 2026). We therefore also compute sliding model-token Distinct-1/2/3 after excluding special tokens. Higher Distinct- n means a larger fraction of unique overlapping n -grams. **XXX: Recompute or replace workbook values labelled “1-gram repetition,” which predate the final Distinct- n definition.**

5 RESULTS

5.1 PRELIMINARY BENCHMARK ACCURACY

Table 4 transcribes the current `Results.xlsx` workbook. Empty cells are deliberately retained as XXX: in particular, several paired AR evaluations are still running. These preliminary values should not yet be used for significance claims.

The provisional complete diffusion macro averages are 56.4% for Gemma, 51.3% for Llama-25k, 48.9% for Llama-50k, 51.3% for Qwen, and 55.1% for LLaDA in our harness. Ministral’s 64.0% and the AR averages are explicitly partial. These means mix 50- and 250-example estimates and

Table 4: Preliminary exact-match/pass@1 accuracy in our matched harness. D: converted diffusion model; AR: unchanged parent. Llama-50k shares the Llama-25k AR parent. Bold marks the best diffusion result in each row; underline marks the best converted LAD-Generic result (ties retained). Thus a bold underlined value is best both overall and among our conversions. † denotes 250 examples and other populated cells use 50. Parenthesized averages cover only the indicated number of populated tasks; XXX is unfinished.

Task	Gemma 25k		D-25k	Llama D-50k		Qwen 25k		Ministral 25k		LLaDA	
	D	AR		D	AR	D	AR	D	AR	Ours	Published
ARC-C	.880	XXX	.828 [†]	.780	.776 [†]	.840	XXX	.820	.740	.880	.885
GPQA	.360	XXX	.280	.220	XXX	.380	XXX	.240	XXX	.300	.333
GSM8K	.600	XXX	.652 [†]	.620	.856 [†]	.560	XXX	.600	.860	.640	.694
HellaSwag	.760	XXX	.732 [†]	.840	.588 [†]	.740	XXX	.820	.720	.720	.746
HumanEval	.500	XXX	.500	.380	XXX	.320	XXX	XXX	XXX	.580	.494
MATH-500	.300	XXX	.220	.180	XXX	.360	XXX	XXX	XXX	.380	.319
MBPP	.600	XXX	.460	.400	XXX	.540	XXX	XXX	XXX	.480	.410
MMLU	.640	.720	.612 [†]	.620	.560 [†]	.600	.660	.600	.520	.560	.655
MMLU-Pro	.440	.460	.332 [†]	.360	.448 [†]	.280	.400	.360	.320	.420	.370
Average	.564	.590 (2)	.513	.489	.646 (5)	.513	.530 (2)	.640 (5)	.632 (5)	.551	.545

Table 5: Published diffusion-model accuracies transcribed from `Results.xlsx`; NR means not reported in the workbook. These values use the respective papers’ protocols and are not directly comparable to Table 4. Bold is the best reported value in a row; underline is the best excluding LLaDA. The final row reproduces each workbook column’s stated average, whose task coverage differs across papers.

Task	LLaDA	DiffuLLaMA	Dream-Inst.	iLLaDA-Inst.	Efficient-DLM 8B
ARC-C	.885	NR	NR	NR	.619
GPQA	.333	NR	.330	NR	NR
GSM8K	.694	NR	.810	.890	.883
HellaSwag	.746	NR	NR	NR	.725
HumanEval	.494	NR	.555	.659	.683
MATH	.319	NR	.392	.567	.501
MBPP	.410	NR	.588	.580	.610
MMLU	.655	NR	.670	.716	.772
MMLU-Pro	.370	NR	.433	.523	NR
Workbook average	.545	NR	.495	.609	.652

are descriptive only. Notably, the second Llama stage does not uniformly improve downstream accuracy despite reducing its 64-step generation perplexity (Section 5.2), motivating checkpoint-wise evaluation rather than selecting solely by denoising loss or fluency.

DiffuLLaMA is included for methodological completeness, but the workbook contains no compatible reported values for these nine instruction benchmarks, so its cells remain NR rather than importing scores from a different task suite. The published averages also have different denominators: the workbook averages all nine LLaDA tasks but excludes GSM8K, as well as unreported cells, for Dream, iLLaDA, and Efficient-DLM. They therefore summarize each column but must not be read as a controlled ranking.

5.2 OPEN-ENDED FLUENCY AND DENOISING BUDGET

Increasing the denoising budget strongly improves this diagnostic. For example, Llama-25k changes from 30.049 at 32 steps to 10.503 at 64 and 3.753 at 128; Gemma-25k changes from 21.795 to 8.017 and 3.317. We call settings with fewer denoising network evaluations than generated tokens ($\text{NFE} < L$) the *parallel decoding regime*; unlike AR decoding, they permit more than one token to be finalized per model evaluation. Perplexity values are meaningful only under matched output length, sampler, temperature, and denoising budget.

Table 6: Token-weighted Phi-4 perplexity on 128-token open-ended generations at 64 denoising steps. Lower is better, but this is a secondary fluency diagnostic.

Model family/checkpoint	Diffusion	AR parent
Gemma 2 9B, 25k	8.017	3.011
Llama 3.1 8B, 25k	10.503	2.489
Llama 3.1 8B, 50k	8.709	2.489
Qwen2.5 7B, 25k	10.272	2.230
Ministral 8B, 25k	9.424	2.305
LLaDA 8B Instruct	4.065	–

Table 7: Training progression: token-weighted Phi-4 perplexity for 128-token generations.

Model	Steps	1k	10k	20k	30k	40k	50k
Llama	32	15.502	23.138	17.091	14.737	17.201	16.797
Llama	64	10.256	10.015	9.646	9.839	8.716	8.382
Llama	128	5.795	5.287	4.763	4.664	4.420	4.876
Model	Steps	1k	5k	10k	15k	20k	25k
Gemma	32	19.206	13.235	11.547	12.413	10.398	11.688
Gemma	64	10.387	7.506	7.941	6.746	6.751	7.826
Gemma	128	5.852	4.457	4.313	3.454	3.941	3.872

We also benchmarked the saved Llama adapter before and after merging under identical 30-prompt, 128-token, 32-step settings (Appendix C). Merging removes 436.2M deployed parameters while changing token-weighted perplexity from 19.381 to 19.283 (−0.51%); Distinct-1/2/3 change from .495/.788/.899 to .498/.807/.918. This matched test provides no evidence of deterioration from merging.

5.3 PROGRESSION WITH TRAINING UPDATES

Table 7 shows checkpoint-wise Phi-4 perplexity from the workbook. Improvements are not monotonic at every budget. The matched final-checkpoint comparison suggests that the second Llama stage primarily helps in the parallel decoding regime: for 128-token outputs, 25k to 50k training lowers perplexity from 30.049 to 16.797 at 32 NFE and from 10.503 to 8.709 at 64 NFE, while the 128-NFE value worsens from 3.753 to 4.876. Meanwhile, the preliminary factual-task macro average changes from 51.3% to 48.9%. Continued training therefore appears to improve low-budget parallel decoding rather than general factual capability; ongoing evaluations will test whether this pattern holds across models and seeds. Gemma reaches most of its 64-step improvement within 15–20k updates and slightly degrades by 25k. Final benchmark results will report fixed checkpoints rather than choosing a separate best checkpoint per task.

5.4 BASE-MODEL DEPENDENCE AND LLADA COMPARISON

All four converted families produce usable diffusion generations, but the resulting quality is not architecture invariant. Gemma currently has the strongest complete diffusion macro average, while Qwen and Llama are similar on average but differ by task. The incomplete Ministral row already illustrates another important point: its strong multiple-choice and GSM8K scores do not establish code or MATH capability until those evaluations finish. **XXX: Replace this paragraph with paired D-minus-AR retention gaps and confidence intervals once every AR cell is populated.**

LLaDA scores 55.1% on the nine-task unweighted average in our harness, compared with 54.5% from the values transcribed as published. Agreement varies by task and should not be interpreted as an exact reproduction because prompts, subsets, generation lengths, answer extraction, and code execution may differ. The final submission will present our common-harness comparison as primary and Table 5 only as external calibration. We chose LLaDA for direct external testing because it uses the uncomplicated response-masking objective closest to ours and reports the largest overlap with our task suite. Dream, iLLaDA, and Efficient-DLM add different data scales, noise weighting, SFT formats, scoring rules, attention patterns, or positional masking. Importing those techniques into the

primary experiment would prevent us from attributing outcomes to parameter-efficient conversion itself.

6 DISCUSSION

The central empirical finding is qualitative but robust across the selected runs: high-rank attention-only LoRA is sufficient to turn four independently developed causal instruction models into generators that reconstruct answers from a fully masked block. This supports the view that much of the knowledge needed for diffusion generation is already present in AR representations and vocabulary heads. Because each conversion fits on one Google Colab GPU rather than requiring a dedicated multi-accelerator cluster, the method also lowers the infrastructure threshold for reproducing, modifying, and teaching large-scale diffusion-language-model conversion. A capable paid cloud GPU is still required, so this is broader access rather than zero-cost access.

The merge result also shows that LoRA’s parameter-count overhead is not permanent: the deployed Llama returns from 8.466B to 8.030B parameters without measurable degradation in this small matched test. This is not model compression—the full base-sized checkpoint remains—and more prompts and backbones are needed to bound small merge effects.

Conversion is not free, however. First, 323–573M trainable parameters are parameter-efficient relative to full-model training but substantially larger than conventional low-rank task adapters. Second, our secondary perplexity remains worse than each AR parent and than LLaDA at 64 steps. Third, additional Llama training does not improve the provisional downstream benchmark aggregate, but its largest fluency gains occur when $NFE < L$. This suggests a more specific benefit: additional denoising training may teach the model to resolve multiple mutually dependent positions per evaluation, improving the quality–compute trade-off without adding corresponding benchmark capability. The 128-NFE result cautions that this interpretation remains preliminary. Finally, EOS-padding supervision makes the fixed-width objective different from a pure answer-only likelihood and deserves explicit ablation.

Perplexity alone is insufficient for selecting a diffusion generator. Our soft-frontier masking ablation lowered Phi-4 generative perplexity but substantially reduced sliding model-token Distinct-1/2/3 and produced visibly more repetitive answers (Appendix D). DiffuLLaMA similarly pairs external-model perplexity with distinct bigrams to check that low perplexity was not obtained through repetition (Gong et al., 2025). More fundamentally, external generative perplexity reflects a separate AR model’s preferences and ignores diversity, so repeated high-probability text can score deceptively well (Turok et al., 2026). Theory likewise distinguishes token-level perplexity from whole-sequence correctness: a masked diffusion model may approach favorable perplexity with few steps while still requiring a step count that scales with sequence length for low sequence error (Feng et al., 2025). We therefore triangulate external perplexity with Distinct-1/2/3, task accuracy, and generated samples. Distinct- n is itself only a repetition diagnostic, so grammaticality, coherence, and actual usability ultimately require systematic qualitative inspection rather than replacement by any single automatic score.

The literature comparison sharpens the trade-off. DiffuLLaMA and Dream reuse AR initialization but update the full model for 65B and 580B tokens, respectively; Efficient-DLM invests 500B tokens and architectural training choices specifically designed to retain causal-model behavior and enable cached block decoding (Gong et al., 2025; Ye et al., 2025; Fu et al., 2025). iLLaDA’s strong reported results come from 12T-token diffusion pretraining plus repeated large-scale SFT, a fundamentally different compute regime (Nie et al., 2026). Fast-dLLM v2 and SDLML show that retaining blockwise or sequential structure can reduce conversion cost while enabling practical caching and variable-length generation (Wu et al., 2025; Liu et al., 2025); our complementary question is how far a fully bidirectional model can be converted with adapter-only updates. Uno is closer to our parameter-efficient philosophy because it also trains LoRA diffusion weights, but its goal is different: an AR verifier guarantees samples from the original AR distribution, whereas LAD-Generics seeks an independently usable fully bidirectional generator (Sahoo et al., 2026). Thus the higher published scores in Table 5 do not isolate a single superior masking objective; they bundle substantially different data budgets and modeling or decoding machinery.

The incremental scale is central to the intended contribution. A 25k LAD-Generic run processes at most 102.4M token positions, versus tens or hundreds of billions of tokens for the full-model conversion systems in Table 1, while leaving most parent parameters frozen. This comparison does not erase the sunk cost of AR pretraining, nor prove a proportional reduction in energy or emissions. It does show that an existing foundation checkpoint can be repurposed without repeating that pretraining or reserving a large cluster. That distinction matters both economically and for widening participation in diffusion-model research; direct power and carbon measurements remain necessary for an environmental comparison (Strubell et al., 2019; Patterson et al., 2021).

This is also why we do not claim that Table 5 is a leaderboard. Our direct LLaDA run holds prompts, subsets, extraction, and generation code fixed. The remaining columns are literature context, useful for locating the attainable performance range but unsuitable for causal comparison. A future study can add these systems to the shared harness one at a time, provided their task-specific inference procedures are faithfully reproduced—a requirement emphasized by the dLLM framework (Zhou et al., 2026).

7 LIMITATIONS

The current workbook combines 50- and 250-example benchmark subsets and does not yet contain every AR baseline. Unless expanded, these estimates have high sampling uncertainty. Only one training seed is available per model, so architecture and random-seed effects are not separated. The four backbones differ in tokenizer, pretraining data, alignment, attention implementation, and nominal parameter count. Although chosen as the latest comparable text-only releases when the study was designed, they are not guaranteed to remain the latest checkpoints at publication time. Phi-4 perplexity favors text that its own AR distribution considers probable, ignores diversity, and is not a diffusion likelihood. Distinct- n detects local repetition but not factuality, grammar, or global coherence. Published LLaDA numbers are not directly comparable to our harness. We also have not yet established wall-clock speedups: $NFE < L$ reduces sequential model calls but does not automatically imply lower latency because each call processes the full answer block. Finally, we did not instrument the runs for energy consumption or carbon intensity. Single-GPU execution and lower incremental training volume support accessibility and compute-efficiency claims, not a quantified sustainability claim.

8 CONCLUSION

LAD-Generic provides a common, parameter-efficient route from instruction-tuned AR checkpoints to masked diffusion generators across Gemma, Llama, Qwen, and Ministral. The preliminary evidence shows broad transfer of the conversion mechanism, but also substantial dependence on the parent model, inference budget, and evaluation metric. By fitting every conversion on a single Google Colab GPU, the work offers a reproducible pathway for researchers who cannot retrain a foundation model or provision a large accelerator cluster. Continued training appears more promising for improving generation in the parallel decoding regime than for improving the aggregate downstream benchmark score, a hypothesis the final checkpoint sweep will test. Completing the paired AR evaluation will determine not merely whether these models can diffuse, but how much of their original capability survives conversion and whether that modest incremental cost is justified relative to purpose-trained diffusion models.

REPRODUCIBILITY STATEMENT

The main text specifies the objective, trainable modules, data volume, optimization, generation, benchmarks, and scoring. The supplementary material will include resolved configurations, parameter audits, immutable code and dataset revisions, checkpoint identifiers, loss logs, generated samples, benchmark manifests, frontier-ablation outputs, and uncertainty calculations. Before submission we will complete the remaining AR cells, archive the historical dataset manifest, report conversion GPU-hours and software versions, and replace every red XXX marker with a measured value or an explicit limitation.

ETHICS STATEMENT

This study uses existing language models and public benchmark or instruction datasets and does not involve human subjects. Converted models inherit factual errors, biases, unsafe behaviors, licensing constraints, and memorization risks from their parent checkpoints and training data. We report aggregate task performance rather than claiming safety or deployment readiness; release artifacts should preserve upstream licenses and document the data provenance and intended research use.

AI USE STATEMENT

Generative AI tools were used to assist software development and debugging, design and review of experiments, creation of analysis scripts, interpretation of preliminary results, literature discovery and summarization, and drafting and editing this manuscript. The authors manually reviewed AI-assisted code and prose, checked numerical claims against saved experimental artifacts, and verified literature claims and citations against primary sources. No benchmark value was entered solely from an unsupported model response. The authors take responsibility for the final content, claims, code, and artifacts.

REFERENCES

- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.
- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, 2021a.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021b.
- Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. In *arXiv preprint arXiv:1803.05457*, 2018.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Rei Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Guhao Feng, Yihan Geng, Jian Guan, Wei Wu, Liwei Wang, and Di He. Theoretical benefit and limitation of diffusion language model. *arXiv preprint arXiv:2502.09622*, 2025.
- Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, Maksim Khadkevich, Jan Kautz, Yingyan Celine Lin, and Pavlo Molchanov. Efficient-DLM: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2025.
- Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Jiawei Han, Hao Peng, and Lingpeng Kong. Scaling diffusion language models via adaptation from autoregressive models. In *International Conference on Learning Representations*, 2025.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021a.

- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems*, 2021b.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Ruurd J. A. Kuiper, Lars de Groot, Bram van Es, Maarten van Smeden, and Ayoub Bagheri. LAD: LoRA-adapted diffusion. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 97–110, 2025.
- Yangzhou Liu, Yue Cao, Hao Li, Gen Luo, Zhe Chen, Weiyun Wang, Xiaobo Liang, Biqing Qi, Lijun Wu, Changyao Tian, Yanting Zhang, Yuqiang Li, Tong Lu, Yu Qiao, Jifeng Dai, and Wenhai Wang. Sequential diffusion language models. *arXiv preprint arXiv:2509.24007*, 2025.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Shen Nie, Qiyang Min, Shaoxuan Xu, Zihao Huang, Yuxuan Song, Yong Shan, Yankai Lin, Wayne Xin Zhao, Chongxuan Li, and Ji-Rong Wen. Improved large language diffusion models. *arXiv preprint arXiv:2606.25331*, 2026.
- David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.
- Subham Sekhar Sahoo, Lingjie Chen, Khiem Pham, Jonathan Geuter, Chaitanya Dwivedi, Varad Pimpalkhute, Yash Akhauri, Alexander Moreno, Mikhail Yurochkin, Zhenting Wang, Mostafa Elhoushi, Nolan Dey, Shane Bergsma, Joel Hestness, John Thickstun, Eric Xing, and Zhengzhong Liu. Unlocking lossless speedups in LLMs via discrete diffusion. *arXiv preprint arXiv:2609.04010*, 2026.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. *arXiv preprint arXiv:2406.04329*, 2024.
- Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650, 2019. doi: 10.18653/v1/P19-1355.
- Gilad Turok, Chris De Sa, and Volodymyr Kuleshov. DUEL: Exact likelihood for masked diffusion via deterministic unmasking. *arXiv preprint arXiv:2603.01367*, 2026.
- Shuaiddi Wang, Zhan Zhuang, Ruping Huang, and Yu Zhang. NaRA: Noise-aware LoRA for parameter-efficient fine-tuning of diffusion LLMs. *arXiv preprint arXiv:2605.29716*, 2026.
- Yubo Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dLLM v2: Efficient block-diffusion LLM. *arXiv preprint arXiv:2509.26328*, 2025.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of ACL*, 2019.

Zhanhui Zhou, Lingjie Chen, Hanghang Tong, and Dawn Song. dLLM: Simple diffusion language modeling. *arXiv preprint arXiv:2602.22661*, 2026.

A RESULT PROVENANCE AND PLACEHOLDER POLICY

Table 4, Table 6, and Table 7 were initially transcribed from `results/Results.xlsx` on September 18, 2026. The open-ended and merge values are archived in `Results_open-ended-128t-32i-merged_20260918.xlsx`. Red-font workbook entries were interpreted as 250-example estimates and black-font entries as 50-example estimates, following the workbook legend. All XXX markers denote information not established by the available artifact; they must be resolved rather than silently inferred before submission.

B SELECTED RUN IDENTIFIERS

The four 25k runs are `gemma-2-9b-mask`, `llama-3.1-8b-mask`, `qwen-2.5-7b-mask`, and `ministral-8b-mask` (the workbook shortens the last name to `ministral-mask`). The second-stage Llama run is `llama-3.1-8b-mask-continued`. The saved configurations identify the parent checkpoints listed in Table 2 and record 25,000 updates for each directory.

The matched open-ended adapter results come from benchmark run `20260918T112818.173051Z--colab-validation`; the merged Llama results come from `20260918T123901.774999Z--colab-validation`. Both use 30 prompts, 128 output tokens, 32 denoising steps, seed 1234, temperature 0.7, the LLaDA-style sampler, and Microsoft Phi-4 scoring.

C ADAPTER-MERGING ABLATION

Table 8: Llama adapter/merge ablation at 128 generated tokens and 32 denoising steps. PPL is token-weighted; Distinct- n is averaged over answers.

Representation	Parameters	PPL↓	D-1↑	D-2↑	D-3↑
Adapter loaded	8.466B	19.381	.495	.788	.899
Merged standalone	8.030B	19.283	.498	.807	.918

D FRONTIER-MASKING ABLATION

The selected main runs use the simpler IID corruption process: after sampling the example-level noise fraction t , each eligible answer or EOS-padding position is independently masked with probability t . In a later ablation, we tested *soft-frontier masking*, intended to resemble partially completed left-to-right text while retaining stochastic exceptions. For a frontier example, masking probability increases sigmoidally with answer position around a frontier determined by t ; the tested configuration used a 0.75 probability of selecting this branch per example, $\epsilon = 0.03$, and $\tau = 3.0$, with the remaining examples using IID corruption. Token losses use the corresponding inverse masking-probability correction.

The frontier variant achieved lower external Phi-4 generative perplexity than the matched IID variant, but substantially lower sliding model-token Distinct-1, Distinct-2, and Distinct-3, accompanied by visibly more immediate token and phrase repetition. This disagreement is precisely why the main experiments retain IID corruption and why generative perplexity is not used as the sole selection criterion. **XXX: Insert the matched run identifiers, checkpoint, generation NFE, token-weighted perplexity, Distinct-1/2/3 values, and representative blinded samples after consolidating the ablation artifacts.**